

# Probe

Zero-Setup Runtime Tuning Console for Unity

Offline User Guide and Script Reference



| Asset               | Probe                                        |
|---------------------|----------------------------------------------|
| Publisher           | HanoStudio                                   |
| Document version    | 1.0.1                                        |
| Last updated        | July 10, 2026                                |
| Unity compatibility | Unity 2021.3 LTS or newer; tested on Unity 6 |

This document is intended for offline use with the Unity Asset Store package.

# Table of Contents

|                                                          |          |
|----------------------------------------------------------|----------|
| <b>1. Overview</b>                                       | <b>3</b> |
| 1.1 What Probe creates                                   | 3        |
| 1.2 Requirements                                         | 3        |
| 1.3 Package contents                                     | 3        |
| <b>2. Setup Guide</b>                                    | <b>3</b> |
| 2.1 Import the package                                   | 3        |
| 2.2 Run the included demo                                | 4        |
| 2.3 Use Probe in your own scene                          | 4        |
| 2.4 Build safety                                         | 4        |
| <b>3. Console Tutorial</b>                               | <b>4</b> |
| 3.1 Open and use the console                             | 4        |
| 3.2 Built-in commands                                    | 5        |
| 3.3 Create a static command                              | 5        |
| 3.4 Create an instance command                           | 5        |
| <b>4. Debug Panel Tutorial</b>                           | <b>5</b> |
| 4.1 Watch live values                                    | 5        |
| 4.2 Add one-tap buttons                                  | 6        |
| 4.3 Add live sliders                                     | 6        |
| 4.4 Add a slider in code                                 | 6        |
| <b>5. Preset Tutorial</b>                                | <b>6</b> |
| 5.1 Quick panel preset                                   | 6        |
| 5.2 Named console presets                                | 6        |
| 5.3 Preset file location                                 | 7        |
| <b>6. Script Reference</b>                               | <b>7</b> |
| 6.1 Attributes                                           | 7        |
| 6.2 Console and commands                                 | 7        |
| 6.3 Panel, buttons, tweaks, and presets                  | 8        |
| 6.4 Supported command arguments                          | 8        |
| <b>7. Troubleshooting</b>                                | <b>8</b> |
| 7.1 The panel or console does not appear                 | 8        |
| 7.2 Instance commands, buttons, or sliders do not appear | 9        |
| 7.3 IL2CPP and managed stripping                         | 9        |
| 7.4 Input backend notes                                  | 9        |
| <b>8. Review Checklist</b>                               | <b>9</b> |

# 1. Overview

Probe is a runtime tuning console for Unity. It creates a small on-screen debug panel and an in-game console during Play Mode, so developers can inspect live values, run commands, tune numeric values, and save tuning presets without adding prefabs or manager objects.

## 1.1 What Probe creates

- A debug panel in the top-left corner showing FPS, memory, custom watches, buttons, sliders, and quick preset controls.
- A console that opens with the backquote key or the on-screen console button.
- Runtime UI built from code with Unity uGUI.
- A sample scene at Assets/Probe/Demo/ProbeDemo.unity.

## 1.2 Requirements

| Requirement     | Details                                                              |
|-----------------|----------------------------------------------------------------------|
| Unity version   | Unity 2021.3 LTS or newer. The package was tested on Unity 6.        |
| Render pipeline | Built-in Render Pipeline, URP, or HDRP.                              |
| Input           | Legacy Input Manager, the new Input System, or both enabled at once. |
| UI              | Unity uGUI. No TextMeshPro setup is required.                        |
| Scene setup     | No prefab or scene object is required for normal use.                |

## 1.3 Package contents

| Path                       | Purpose                                                           |
|----------------------------|-------------------------------------------------------------------|
| Assets/Probe/Runtime       | Core console, panel, command, action, tweak, preset, and UI code. |
| Assets/Probe/Demo          | Optional sample scene and ProbeDemo component.                    |
| Assets/Probe/README.md     | Short text guide for users who prefer Markdown.                   |
| Assets/Probe/Documentation | This offline PDF guide.                                           |

# 2. Setup Guide

## 2.1 Import the package

1. Open your Unity project.
2. Import the Probe package through Assets > Import Package > Custom Package, or import it from the Unity Asset Store.
3. Confirm that the Assets/Probe folder is present in the Project window.
4. Open any scene and press Play.
5. The debug panel should appear in the top-left corner automatically in the Editor.

No prefab, component, or scene object is required for Probe to start in the Editor or in Development builds.

## 2.2 Run the included demo

1. Open Assets/Probe/Demo/ProbeDemo.unity.
2. Press Play.
3. Confirm that the debug panel appears in the top-left corner.
4. Use the on-screen buttons, such as Full Heal, Hurt 10, and Toggle Slomo.
5. Drag the bobSpeed and bobHeight sliders to change the demo object while the scene is running.
6. Open the console with the backquote key or the on-screen console button.
7. Type help, then try heal 25, hurt 10, teleport 2 0 5, slomo 0.3, and echo "hi there".

## 2.3 Use Probe in your own scene

1. Import the package.
2. Press Play in your scene. The default panel appears automatically in the Editor and Development builds.
3. Add watches with ProbePanel.Watch when you want project-specific values on the panel.
4. Add DebugCommand, DebugButton, or DebugTweak attributes to your own code as needed.
5. Register live MonoBehaviour instances when you use instance methods or instance fields.

## 2.4 Build safety

Probe auto-runs only when UNITY\_EDITOR, DEVELOPMENT\_BUILD, or PROBE\_ENABLE is defined. It does not create the console or panel in a normal release build unless you opt in. PROBE\_DISABLE takes priority and turns Probe off everywhere.

At play-mode startup, Probe resets its static command, button, tweak, panel, console, and UI cache state. This prevents stale values or event subscribers from carrying between Play sessions when Domain Reload is disabled for faster Enter Play Mode.

| Define symbol | Effect                                              |
|---------------|-----------------------------------------------------|
| None          | Probe runs in the Editor and in Development builds. |
| PROBE_ENABLE  | Force Probe on, including non-development builds.   |
| PROBE_DISABLE | Force Probe off everywhere.                         |

# 3. Console Tutorial

## 3.1 Open and use the console

- Press the backquote key to show or hide the console.
- Use the on-screen console button if the keyboard is not convenient.
- Type help to list commands.
- Type help command\_name to see usage for one command.
- Use Tab to autocomplete command names.
- Use Up Arrow and Down Arrow to recall command history.

## 3.2 Built-in commands

| Command      | Description                                  |
|--------------|----------------------------------------------|
| help         | Lists all registered commands.               |
| help         | Shows usage and description for one command. |
| echo         | Prints text back to the console.             |
| preset_save  | Saves current DebugTweak slider values.      |
| preset_load  | Loads saved DebugTweak values.               |
| preset_list  | Lists saved presets.                         |
| preset_reset | Resets tweaks to their startup values.       |

## 3.3 Create a static command

Static commands are discovered automatically. Use supported parameter types such as string, int, float, double, bool, and enum. Optional parameters with default values are supported.

```
using Probe;

public static class Cheats
{
    [DebugCommand("give_gold", "Add gold to the player")]
    static string GiveGold(int amount)
    {
        Player.Gold += amount;
        return $"Gold is now {Player.Gold}";
    }
}
```

## 3.4 Create an instance command

Instance commands require a live object registration. Register in Awake or Start, and unregister in OnDestroy.

```
using Probe;
using UnityEngine;

public class PlayerDebug : MonoBehaviour
{
    void Awake() => CommandRegistry.RegisterInstance(this);
    void OnDestroy() => CommandRegistry.UnregisterInstance(this);

    [DebugCommand("heal", "Heal the player")]
    string Heal(int amount)
    {
        health = Mathf.Min(maxHealth, health + amount);
        return $"HP = {health}/{maxHealth}";
    }
}
```

# 4. Debug Panel Tutorial

## 4.1 Watch live values

Use `ProbePanel.Watch` to display live values below the built-in FPS and memory lines. Calling `Watch` again with the same label replaces the getter.

```
ProbePanel.Watch("HP", () => $"{health}/{maxHealth}");
ProbePanel.Watch("State", () => currentState);
```

```
ProbePanel.Unwatch("State");
```

## 4.2 Add one-tap buttons

Mark a parameterless method with `DebugButton`. Static buttons are discovered automatically. Instance buttons require `ProbeActions.RegisterInstance`.

```
[DebugButton("Full Heal")]
void FullHeal()
{
    health = maxHealth;
}

void Awake() => ProbeActions.RegisterInstance(this);
void OnDestroy() => ProbeActions.UnregisterInstance(this);
```

## 4.3 Add live sliders

Mark a numeric field or property with `DebugTweak` to create a slider. Supported member types are float, int, double, and long. Instance members require `ProbeTweaks.RegisterInstance`.

```
[DebugTweak(0f, 20f, "Move Speed")]
public float moveSpeed = 5f;

[DebugTweak(1, 10)]
public int jumpCount = 2;

void Awake() => ProbeTweaks.RegisterInstance(this);
void OnDestroy() => ProbeTweaks.UnregisterInstance(this);
```

## 4.4 Add a slider in code

```
ProbeTweaks.Add(
    "Gravity",
    -30f,
    0f,
    () => Physics.gravity.y,
    value => Physics.gravity = new Vector3(0f, value, 0f));
```

# 5. Preset Tutorial

## 5.1 Quick panel preset

1. Create at least one `DebugTweak` slider.
2. Press Play.
3. Adjust the slider values.
4. Click Save on the debug panel to store the quick preset.
5. Change the values again.
6. Click Load to restore the quick preset, or Reset to return to startup values.

## 5.2 Named console presets

```
preset_save floaty_jump
preset_load floaty_jump
preset_list
preset_reset
```

## 5.3 Preset file location

Named presets are JSON files saved under `Application.persistentDataPath/ProbePresets`. Presets address tweaks by label, so keep `DebugTweak` labels stable when sharing presets between runs or devices.

# 6. Script Reference

## 6.1 Attributes

| API                                | Use                                                                                                                                            |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>DebugCommandAttribute</code> | Marks a method as a console command. Constructor: <code>DebugCommandAttribute(string name, string description = "")</code> .                   |
| <code>DebugButtonAttribute</code>  | Marks a parameterless method as a panel button. Constructor: <code>DebugButtonAttribute(string label = null)</code> .                          |
| <code>DebugTweakAttribute</code>   | Marks a numeric field or property as a live slider. Constructor: <code>DebugTweakAttribute(float min, float max, string label = null)</code> . |

## 6.2 Console and commands

| API                                                            | Use                                                                   |
|----------------------------------------------------------------|-----------------------------------------------------------------------|
| <code>CommandRegistry.Commands</code>                          | Read-only dictionary of registered commands.                          |
| <code>CommandRegistry.EnsureBuilt()</code>                     | Forces static command discovery.                                      |
| <code>CommandRegistry.RegisterInstance(object target)</code>   | Registers instance <code>DebugCommand</code> methods.                 |
| <code>CommandRegistry.UnregisterInstance(object target)</code> | Removes commands for an instance.                                     |
| <code>CommandParser.Execute(string input)</code>               | Parses and executes one console command line and returns output text. |
| <code>ProbeConsole.Instance</code>                             | Current console instance, if created.                                 |
| <code>ProbeConsole.Toggle()</code>                             | Shows or hides the console instance.                                  |
| <code>ProbeConsole.SetVisible(bool visible)</code>             | Sets console visibility.                                              |
| <code>ProbeConsole.IsVisible</code>                            | True when the console is open.                                        |

## 6.3 Panel, buttons, tweaks, and presets

| API                                            | Use                                                          |
|------------------------------------------------|--------------------------------------------------------------|
| ProbePanel.Watch(string label, Func getter)    | Adds or replaces a live panel value.                         |
| ProbePanel.Unwatch(string label)               | Removes a watched value.                                     |
| ProbePanel.Toggle()                            | Toggles the panel user's requested visibility.               |
| ProbePanel.SetVisible(bool visible)            | Sets the panel user's requested visibility.                  |
| ProbeActions.RegisterInstance(object target)   | Registers instance DebugButton methods.                      |
| ProbeActions.UnregisterInstance(object target) | Removes buttons for an instance.                             |
| ProbeTweaks.RegisterInstance(object target)    | Registers instance DebugTweak members.                       |
| ProbeTweaks.UnregisterInstance(object target)  | Removes tweaks for an instance.                              |
| ProbeTweaks.Add(...)                           | Adds a programmatic slider with getter and setter callbacks. |
| ProbePresets.Save(string name)                 | Saves current tweak values to a named preset.                |
| ProbePresets.Load(string name)                 | Loads a named preset if present.                             |
| ProbePresets.Reset()                           | Restores all tweaks to their registration-time defaults.     |
| ProbePresets.List()                            | Returns saved preset names.                                  |
| ProbePresets.Delete(string name)               | Deletes a named preset if present.                           |

## 6.4 Supported command arguments

| Type   | Notes                                     |
|--------|-------------------------------------------|
| string | Quoted strings with spaces are supported. |
| int    | Parsed as an integer.                     |
| float  | Parsed with invariant culture.            |
| double | Parsed with invariant culture.            |
| bool   | Supports true, false, 1, and 0.           |
| enum   | Parsed by name, case-insensitive.         |

# 7. Troubleshooting

## 7.1 The panel or console does not appear

- Confirm that you are running in the Unity Editor or a Development build.
- Check that PROBE\_DISABLE is not defined.
- If you intentionally need Probe in a release build, add PROBE\_ENABLE.



- Confirm that Assets/Probe/Runtime is present and compiles without errors.

## 7.2 Instance commands, buttons, or sliders do not appear

- Register the live object with `CommandRegistry.RegisterInstance`, `ProbeActions.RegisterInstance`, or `ProbeTweaks.RegisterInstance`.
- Unregister the object in `OnDestroy` to avoid stale references.
- For `DebugButton`, confirm the method has no parameters.
- For `DebugTweak`, confirm the member is a readable and writable numeric field or property.

## 7.3 IL2CPP and managed stripping

If you force-enable Probe in a release build and use IL2CPP with managed stripping, members reached only through reflection can be stripped. Add `UnityEngine.Scripting.Preserve` to those members or provide a `link.xml` file that preserves the relevant assembly. The default Editor and Development build workflow is not affected.

## 7.4 Input backend notes

Probe checks the new Input System when `ENABLE_INPUT_SYSTEM` is defined and checks the legacy Input Manager when `ENABLE_LEGACY_INPUT_MANAGER` is defined. Projects with both systems enabled can use either path.

# 8. Review Checklist

1. Import the package.
2. Open Assets/Probe/Demo/ProbeDemo.unity.
3. Press Play and confirm the debug panel appears at the top-left.
4. Open the console and run help.
5. Run heal 25, hurt 10, teleport 2 0 5, and slomo 0.3.
6. Drag the bobSpeed and bobHeight sliders.
7. Use preset\_save demo and preset\_load demo.
8. Confirm the package compiles without external setup.

Support contact: [hellohanostudio@gmail.com](mailto:hellohanostudio@gmail.com)